Sistema de Gestión de Inventarios

JAK Inventory

Ingeniería de Software II — Entrega 1: Investigación y fundamentación

Jasmin Camila Gutierrez Maya
Luz Amanda Galeano Ceballos
Kevvin Estiben Hernandez Jaramillo

Universidad Nacional de Colombia — Sede Manizales
Administración de sistemas informáticos
28 de septiembre de 2026

Repositorio: https://github.com/lugaleanoc/gestion-inventarios-isw2.git

Tabla de contenido

1. Investigación de sistemas similares 2
1.1 Sistemas analizados 2
1.2 Cuadro comparativo 2
2. Definición del problema y alcance 3
2.1 Problema 3
2.2 Alcance del sistema a construir 3
3. Justificación de la arquitectura de microservicios 4
4. Identificación preliminar de microservicios 5
5. Requerimientos funcionales y no funcionales 7
5.1 Requerimientos funcionales (RF) 7
5.2 Requerimientos no funcionales (RNF) 9
6. Bitácora de uso de IA 10

1 Investigación de sistemas similares

1.1 Sistemas analizados

Criterio	Sistema 1: Comercial (Alegra)	Sistema 2: Código Abierto (Odoo)	Sistema 3: Corporativo (Zoho Inventory)
Gestión de productos	Básico: Nombre, precio, categoría y foto. Orientado a comercio tradicional y servicios.	Avanzado: Variantes (tallas/colores), números de serie, lotes y listas de materiales (BOM).	Completo: Atributos de producto, agrupaciones, asignación de SKU y códigos de barras.
Control de stock multi-bodega	Permite crear varias bodegas con transferencias sencillas de mercancía.	Avanzado: Control de ubicaciones físicas (estantes/pasillos), rutas de movimiento y reglas internas.	Flexible: Maneja múltiples bodegas y asigna existencias por canal de venta.
Órdenes de compra	Creación manual de órdenes de compra a proveedores y conversión a factura.	Automático: Detecta faltantes y genera la orden de compra según reglas de reabastecimiento.	Práctico: Genera órdenes de compra rápidas al alcanzar el punto de reorden configurado.
Alertas de stock	Avisos sencillos en pantalla cuando el producto alcanza el nivel mínimo.	Alertas automáticas programables que pueden disparar compras o producción.	Notificaciones automáticas en tiempo real al llegar al umbral de reorden.
Reportes / dashboard	Panel visual sencillo con métricas de ventas, rotación básica y valor del inventario.	Reportes analíticos profundos de valoración de inventario (FIFO/Promedio), rotación y trazabilidad.	Dashboards claros sobre rendimiento de ventas, estado de envíos y productos estancados.
Arquitectura identificada	Nube centralizada (SaaS).	Monolito Modular (Servidor central ejecutable).	Nube basada en Microservicios / Servicios Web.
Experiencia de usuario	Muy amigable, limpia e intuitiva. No requiere capacitación previa.	Funcional pero saturada de opciones y menús. Requiere capacitación.	Interfaz muy moderna, fluida y rápida en PC y dispositivos móviles.

1.2 Cuadro comparativo

Criterio	Comercial (Alegra)	Código Abierto (Odoo)	Corporativo (Zoho Inventory)
Tipo de Sistema	Comercial / SaaS (Pago por suscripción)	Código Abierto (Open Source) / Opción Comercial	Corporativo / Comercial de Nicho (SaaS)
Funcionalidades Principales	Registro de productos, facturación electrónica, control básico de entradas/salidas y reportes de ventas.	Gestión multialmacén, lectura de códigos de barras, trazabilidad por lotes/series y reabastecimiento automático.	Sincronización en tiempo real con tiendas virtuales, alertas de stock mínimo, gestión de compras y despachos.
Arquitectura Identificada	Nube centralizada (SaaS).	Monolito Modular (Servidor central único en Python y PostgreSQL).	Nube basada en Microservicios / Servicios Web independientes.
Fortalezas	Extremadamente fácil de usar, interfaz muy limpia e integración nativa con facturación electrónica.	Súper completo, alta flexibilidad y capacidad de adaptarse a reglas de negocio muy específicas.	Muy visual, rápido de implementar y excelente para ventas multicanal por internet.
Debilidades	Limitado para inventarios avanzados (sin trazabilidad de lotes, fechas de vencimiento ni ensambles).	Configuración inicial compleja y servidores costosos si se requiere personalizar o escalar el monolito.	Sistema rígido que no permite modificar el código fuente si la empresa tiene procesos manuales no convencionales.

2 Definición del problema y alcance

2.1 Problema

Actualmente, muchos negocios que realizan ventas tanto en tiendas físicas como por internet gestionan su inventario y sus movimientos de venta mediante procesos separados, manuales o herramientas que no están completamente integradas. Esta situación puede generar diferencias entre el inventario real y el registrado en el sistema, ocasionando problemas como la venta de productos que ya no se encuentran disponibles, retrasos en la actualización del stock y dificultad para identificar oportunamente los productos que necesitan ser reabastecidos.

Además, cuando un producto alcanza niveles bajos o se agota, la solicitud de reposición al proveedor suele depender de una revisión manual. Esto puede retrasar el proceso de abastecimiento y provocar períodos en los que el negocio no dispone de determinados productos para la venta.

Por otra parte, los negocios que cuentan con varios locales y diferentes canales de venta necesitan consolidar diariamente información como ingresos, egresos y ventas realizadas en efectivo, tarjetas, códigos QR y tienda web. Cuando esta información se registra de manera independiente, el responsable debe realizar cálculos y cruces manuales para conocer cuánto dinero se encuentra disponible en caja y cuánto corresponde a cada medio de pago, aumentando el riesgo de errores y dificultando el seguimiento de las operaciones.

A esto se suma que algunas soluciones existentes ofrecen funcionalidades generales y configuraciones predeterminadas que no necesariamente se adaptan a la forma particular de operar de cada empresa. Esto puede limitar la personalización de los indicadores, alertas y procesos que cada negocio necesita para controlar sus operaciones.

2.2 Alcance del sistema a construir

El sistema estará enfocado principalmente en la gestión centralizada del inventario y el control operativo de las ventas, sin incluir módulos completos de contabilidad, nómina o facturación electrónica. Dentro de este alcance, el sistema va a:

- Manejar el inventario de forma centralizada para las ventas en tienda física y en la tienda web, de modo que el stock se descuente en tiempo real sin importar por cuál canal se realizó la venta.
- Generar automáticamente una solicitud de reposición al proveedor cuando el stock de un producto llegue a cero o alcance un umbral mínimo configurado, reduciendo la necesidad de realizar revisiones manuales.
- Permitir que cada empresa personalice su panel de control, definiendo las métricas que desea visualizar, las alertas que desea activar y la forma de organizar sus productos, en lugar de imponer una única configuración para todos los negocios.
- Estar construido sobre una arquitectura en la nube basada en microservicios o servicios web independientes, permitiendo que funciones como inventario, ventas, reposición, balance diario y reportes puedan desarrollarse, desplegarse y escalarse de manera independiente.
- Incluir dashboards de seguimiento, donde se pueda visualizar el rendimiento de las ventas por canal, el estado de los envíos y los productos con poca o ninguna rotación, facilitando la toma de decisiones sin necesidad de cruzar información manualmente.

- Incluir un microservicio de balance diario, encargado de registrar y consolidar los ingresos y egresos de cada local, así como las ventas realizadas a través de la tienda web. Este microservicio permitirá generar un reporte diario que muestre el dinero disponible según el medio de pago, diferenciando entre efectivo en caja, pagos con tarjeta y pagos mediante códigos QR, facilitando el control de los ingresos de cada establecimiento y de las ventas realizadas por internet.
- Generar reportes del balance diario por local y canal de venta, permitiendo consultar cuánto dinero ingresó, cuánto salió y cuál es el valor disponible en cada medio de pago al finalizar la jornada.

Fuera de alcance

Queda fuera de alcance para este semestre la implementación de pasarelas de pago, facturación electrónica ante la DIAN, contabilidad financiera, nómina y otros módulos administrativos que no estén directamente relacionados con el control de inventario, ventas, reabastecimiento y balance operativo diario.

3 Justificación de la arquitectura de microservicios

Para nuestro sistema evaluamos dos arquitecturas posibles: un monolito, donde todo el sistema (inventario, ventas, proveedores y reportes) corre como una sola aplicación con una sola base de datos; y microservicios, donde cada una de esas funciones es un servicio independiente con su propia base de datos y su propio ciclo de despliegue.

La decisión la basamos en cinco criterios técnicos concretos: escalabilidad, mantenibilidad, capacidad de despliegue independiente, tolerancia a fallos, y organización del trabajo en equipo. En un monolito, estos cinco puntos están limitados porque todo el sistema se comporta como una sola unidad: escala completo aunque solo un módulo lo necesite, se despliega completo aunque solo un módulo haya cambiado, y falla completo aunque el error esté en una sola parte. En microservicios, cada uno de estos aspectos se resuelve a nivel de servicio individual, lo que se ajusta mejor a las necesidades específicas de nuestro sistema (detalladas en la tabla comparativa a continuación).

Criterio técnico	Monolito	Microservicios	Por qué pesa más para este caso
Escalabilidad	Todo el sistema escala como un solo bloque, aunque solo un módulo tenga más carga.	Cada servicio escala por separado según su demanda.	Las ventas en tienda web pueden tener picos fuertes (promociones, fechas especiales) mientras que el módulo de reportes casi no cambia de carga; con microservicios solo escalan el servicio de ventas/inventario, no todo el sistema completo.
Mantenibilidad	Un cambio en un módulo obliga a probar y redesplegar todo el sistema.	Cada servicio se modifica y despliega sin afectar a los demás.	Como el sistema va a tener personalización por empresa (paneles distintos), van a necesitar ajustar el servicio de dashboard/reportes con frecuencia sin arriesgar el servicio de inventario o de compras a proveedores.

Criterio técnico	Monolito	Microservicios	Por qué pesa más para este caso
Despliegue independiente	Un solo despliegue para toda la aplicación; un error en un módulo tumba todo.	Cada servicio se despliega sólo, en su propio momento.	El servicio de "solicitud automática a proveedores" es crítico y debe estar siempre disponible; si estuviera junto al resto en un monolito, un error en, por ejemplo, el dashboard podría tumbar también la lógica de reabastecimiento.
Tolerancia a fallos	Si un módulo falla, puede caerse toda la aplicación.	Un servicio caído no tumba a los demás; el sistema sigue funcionando parcialmente.	Si el servicio de dashboards falla por alguna razón, el negocio necesita que las ventas en tienda y la generación de solicitudes a proveedores sigan funcionando sin interrupción; eso no se garantiza en un monolito.
Organización del equipo de trabajo	Todo el equipo trabaja sobre el mismo código base, lo que genera choques al editar los mismos archivos.	Cada integrante puede encargarse de un servicio distinto de forma independiente.	Somos 3 integrantes: si cada uno se encarga de un grupo de microservicios (por ejemplo, el flujo de inventario y reposición, el de ventas y balance diario, y el de gateway y reportes), pueden trabajar en paralelo sin pisarse el código, algo coherente con el flujo de ramas que ya definimos en Git.

4 Identificación preliminar de microservicios

Microservicio	Dominio	Bounded Context	Responsabilidad principal	Funciones principales
API Gateway (con autenticación integrada)	Entrada al sistema y seguridad	Acceso y autenticación	Ser el único punto de entrada, enrutar las peticiones y validar la identidad y los permisos de quien las realiza.	• Enrutar las peticiones `/api/<servicio>` hacia el microservicio correspondiente. • Registrar y administrar usuarios y sus credenciales (con almacenamiento seguro de contraseñas). • Asignar roles y permisos (ej. administrador, vendedor, encargado de bodega). • Autenticar usuarios (login) y generar y validar tokens de acceso (JWT). • Autenticar las tiendas web externas como servicio (rol `tienda_web`). • Restringir acciones según el rol (ej. solo un rol puede aprobar reposiciones o cambiar configuración). • Contar con su propia base de datos de usuarios.
Proveedores	Gestión de proveedores	Proveedores	Administrar la información de los proveedores de la empresa.	• Registrar y actualizar proveedores. • Consultar datos de contacto y condiciones de cada proveedor. • Asociar proveedores a productos específicos. • Exponer esta información al microservicio de Reposición para que pueda generar las solicitudes.
Inventario	Gestión de inventario	Inventario	Administrar los productos, las bodegas y controlar las existencias disponibles en cada local.	• Registrar y actualizar productos y categorías. • Administrar bodegas y controlar el stock por local. • Consultar disponibilidad. • Registrar entradas, salidas, ajustes, traslados y devoluciones. • Mantener el kardex (histórico de movimientos). • Actualizar existencias cuando Ventas lo solicita. • Identificar los productos que alcanzan el stock mínimo (consultando la configuración) y avisar a Reposición.

Microservicio	Dominio	Bounded Context	Responsabilidad principal	Funciones principales
Ventas	Gestión de ventas	Ventas	Registrar y gestionar las ventas realizadas en los locales físicos y en la tienda web.	• Registrar ventas. • Identificar el canal de venta. • Registrar productos y cantidades vendidas. • Registrar el medio de pago. • Solicitar al microservicio de Inventario la verificación y el descuento del stock. • Enviar la información de cada venta al Balance Diario.
Reposición	Abastecimiento y reposición	Reposición y abastecimiento	Detectar las necesidades de abastecimiento y gestionar las solicitudes y órdenes de reposición a proveedores.	• Consultar niveles de inventario. • Identificar productos agotados o por debajo del mínimo. • Generar solicitudes de reposición cuando stock actual $\leq$ stock mínimo, evitando duplicados. • Asociar solicitudes con proveedores. • Gestionar órdenes de compra y registrar la recepción de mercancía. • Consultar el estado de las solicitudes.
Balance Diario	Control operativo de ingresos y egresos	Balance y movimientos diarios	Consolidar los movimientos económicos operativos del día, sin realizar funciones propias de un sistema contable.	• Registrar ingresos por local. • Registrar egresos por local. • Recibir información de las ventas. • Diferenciar efectivo, tarjetas y QR. • Registrar ventas de la tienda web. • Generar el balance diario. • Mostrar valores por medio de pago. • Generar reportes por local y fecha.
Reportes y Dashboard	Análisis y visualización de información	Reportes y visualización	Presentar de manera visual la información obtenida de los demás microservicios para facilitar el seguimiento de la operación.	• Mostrar ventas por canal. • Mostrar productos con poca rotación. • Mostrar productos agotados. • Mostrar estado de reposiciones. • Mostrar balance diario. • Mostrar ingresos y egresos. • Mostrar información por local. • Generar indicadores y reportes. • Mantener una caché propia con la información consultada.
Configuración	Personalización empresarial	Configuración empresarial	Permitir que cada empresa adapte determinadas reglas, alertas y elementos visuales del sistema.	• Configurar el stock mínimo. • Activar o desactivar alertas. • Seleccionar métricas del dashboard. • Configurar preferencias de visualización. • Administrar configuraciones propias de cada empresa.

Mecanismo de comunicación entre microservicios

Microservicio	Se comunica con	Mecanismo	Justificación
API Gateway	Todos los servicios	REST	Se utiliza REST porque el Gateway recibe las peticiones externas, valida el token y el rol del usuario o de la tienda web, y las enruta de inmediato al microservicio correspondiente, esperando su respuesta para devolverla al cliente.
Proveedores	Reposición, Inventario	REST	Se utiliza REST porque Reposición e Inventario necesitan consultar información actualizada de proveedores, como datos de contacto, condiciones y proveedores asociados a productos, y requieren una respuesta inmediata para continuar con la operación.
Inventario	Ventas, Reposición, Configuración, Reportes	REST	Se utiliza REST porque Ventas solicita verificar y descontar existencias y necesita saber de inmediato si la operación fue posible; Inventario avisa a Reposición cuando un producto alcanza el stock mínimo, consulta a Configuración el valor mínimo vigente y expone sus consultas a Reportes.
Ventas	Inventario, Balance Diario	REST	Se utiliza REST para consultar y descontar el stock en Inventario antes de confirmar una venta, y para registrar el ingreso de cada venta en el Balance Diario.

Microservicio	Se comunica con	Mecanismo	Justificación
Reposición	Inventario, Proveedores, Configuración	REST	Se utiliza REST para consultar el stock actual, la información del proveedor asociado y la configuración vigente, y para registrar en Inventario la entrada de la mercancía cuando se recibe una orden.
Balance Diario	Ventas, Reportes	REST	Se utiliza REST para recibir de Ventas el registro de los ingresos y para responder las consultas de un período determinado que realiza Reportes.
Reportes y Dashboard	Inventario, Ventas, Reposición, Balance Diario, Configuración	REST	Se utiliza REST porque este microservicio principalmente necesita consultar y obtener información para construir indicadores, reportes y dashboards, y la configuración de métricas de cada empresa. Las consultas requieren una respuesta con los datos solicitados.
Configuración	Inventario, Reposición, Reportes	REST	Se utiliza REST porque los demás servicios necesitan consultar las configuraciones actuales, como stock mínimo, alertas y preferencias, antes de ejecutar determinadas operaciones.

Para evitar que la falla de un servicio afecte a los demás, cada llamada REST entre servicios usa circuit breaker: si el servicio destino no responde, se corta la espera y se devuelve un error controlado. Las llamadas fallidas se reintentan hasta tres veces antes de registrarse como fallo (ver RF30).

5 Requerimientos funcionales y no funcionales

5.1 Requerimientos funcionales (RF)

RF01 Gestión de productos: El sistema debe permitir crear, consultar, actualizar y eliminar (dar de baja) productos, incluyendo atributos como código, nombre, descripción, categoría, unidad de medida, precio y proveedor asociado.

RF02 Gestión de categorías: El sistema debe permitir crear, consultar, actualizar y eliminar categorías y subcategorías de productos.

RF03 Gestión de proveedores: El sistema debe permitir registrar, consultar, actualizar y eliminar proveedores, y asociarlos a uno o varios productos.

RF04 Control de stock: El sistema debe permitir registrar movimientos de inventario (entradas, salidas, ajustes y traslados) y mantener actualizado el saldo disponible de cada producto en cada bodega o almacén.

RF05 Gestión de almacenes o bodegas: El sistema debe permitir administrar múltiples ubicaciones físicas de almacenamiento y consultar el stock de un producto por ubicación.

RF06 Órdenes de compra: El sistema deberá permitir generar, consultar y actualizar el estado de las órdenes de compra asociadas a solicitudes de reposición, así como registrar la recepción de los productos solicitados.

RF07 Gestión de usuarios y roles: El sistema debe permitir, a través del módulo de autenticación integrado en el API Gateway, registrar, consultar, actualizar y asignar usuarios, con roles y permisos diferenciados.

RF08 Autenticación de usuarios: El API Gateway debe permitir el inicio de sesión, cierre de sesión, recuperación de contraseña y cambio de contraseña, y emitir los tokens de acceso (JWT) que validan las peticiones.

RF09 Alertas de stock: El sistema debe notificar, mediante una alerta visible en el panel web, cuando el nivel de stock de un producto esté por debajo de un umbral mínimo configurable.

RF10 Trazabilidad y auditoría: El sistema debe registrar quién, cuándo y qué movimiento realizó sobre el inventario, manteniendo un histórico de movimientos o kardex.

RF11 Reportes: El sistema debe generar reportes básicos de inventario, incluyendo existencias actuales, valorización del inventario y productos con mayor y menor rotación.

RF12 Búsqueda y filtrado: El sistema debe permitir buscar y filtrar productos por nombre, código, categoría o proveedor.

RF13 Gestión de devoluciones: El sistema debe permitir registrar devoluciones de productos, tanto de clientes como hacia proveedores, y reflejarlas en el stock.

RF14 Panel de indicadores: El sistema debe presentar indicadores clave del inventario, como rotación, quiebres de stock y valor total.

RF15 Registro de ventas por canal: El sistema debe registrar las ventas indicando el canal por el cual se realizaron, ya sea tienda física o tienda web.

RF16 Registro del medio de pago: El sistema debe registrar el medio de pago utilizado en cada venta, incluyendo efectivo, tarjeta o código QR.

RF17 Actualización de stock por venta: El sistema deberá actualizar el stock correspondiente en el microservicio de Inventario después de registrar una venta, mediante una llamada REST del microservicio de Ventas al de Inventario.

RF18 Generación automática de reposición: El sistema deberá generar automáticamente una solicitud de reposición cuando el stock disponible de un producto sea menor o igual al stock mínimo configurado para la empresa.

RF19 Consulta de solicitudes de reposición: El sistema debe permitir consultar el estado de una solicitud de reposición, por ejemplo: pendiente, aprobada o recibida.

RF20 Asociación con proveedores: El sistema debe asociar cada solicitud de reposición con el proveedor correspondiente al producto, consultando dicha información en el microservicio de Proveedores.

RF21 Registro y consolidación de movimientos: El sistema debe registrar y consolidar los ingresos y egresos de cada local, diferenciando entre efectivo, tarjeta y pagos QR.

RF22 Registro de ventas web en el balance: El sistema debe incluir en el balance diario las ventas realizadas por la tienda web.

RF23 Reportes de balance: El sistema debe generar reportes del balance diario filtrables por local y por fecha.

RF24 Indicadores operativos: El sistema debe presentar indicadores de ventas por canal, productos con poca o ninguna rotación, productos agotados y estado de las reposiciones.

RF25 Información consolidada por local: El sistema debe permitir visualizar información consolidada de inventario, ventas y balance, segmentada por local.

RF26 Configuración de stock mínimo: El sistema debe permitir que cada empresa configure su propio stock mínimo por producto.

RF27 Configuración de alertas: El sistema debe permitir activar o desactivar alertas de forma independiente.

RF28 Configuración del dashboard: El sistema debe permitir que cada empresa seleccione las métricas que desea mostrar en su dashboard y ajuste sus preferencias de visualización.

RF29 Control de acceso por roles: El sistema debe restringir acciones críticas, como aprobar una reposición o modificar la configuración de la empresa, según el rol asignado al usuario autenticado, por ejemplo: administrador, vendedor o encargado de bodega.

RF30 Registro y notificación de fallos entre servicios: El sistema debe registrar el resultado de cada llamada REST entre microservicios y notificar al usuario responsable únicamente cuando falle, indicando el servicio de origen, el servicio de destino y la causa. Las llamadas fallidas se reintentan hasta 3 veces; si el error persiste, el fallo queda registrado para su revisión. Los resultados exitosos solo se registran en el log.

5.2 Requerimientos no funcionales (RNF)

RNF01 Arquitectura: El sistema debe estar construido bajo una arquitectura de microservicios, con servicios independientes y débilmente acoplados, comunicados mediante API REST.

RNF02 Persistencia independiente: Cada microservicio debe gestionar su propia base de datos, evitando el acceso directo de un servicio a la base de datos de otro.

RNF03 Contenerización: Todos los microservicios y sus dependencias, como bases de datos, deben ejecutarse en contenedores Docker.

RNF04 Orquestación: El sistema debe desplegarse y gestionarse obligatoriamente mediante Kubernetes.

RNF05 Escalabilidad: La arquitectura debe permitir escalar de forma independiente los microservicios con mayor carga.

RNF06 Rendimiento: El sistema debe responder a las operaciones de consulta en un tiempo menor a 2 segundos bajo una carga concurrente definida y sustentada por el equipo.

RNF07 Seguridad: Las comunicaciones y la autenticación, tanto de usuarios como de tiendas web, deben protegerse mediante mecanismos estándar, como JWT/OAuth2 y mecanismos seguros para el almacenamiento de contraseñas.

RNF08 Documentación de API: Cada microservicio debe exponer documentación de su API mediante OpenAPI, Swagger o una herramienta equivalente.

RNF09 API Gateway propio: Todas las peticiones dirigidas al sistema deben pasar obligatoriamente por un API Gateway construido por el equipo, que además integra la autenticación y la validación de roles.

RNF10 Observabilidad: El sistema debe contar con registro (logging) centralizado o consultable de cada microservicio y, de forma deseable, métricas básicas de monitoreo.

RNF11 Pruebas de rendimiento: Cada integrante del equipo debe definir y ejecutar un mínimo de quince (15) pruebas de rendimiento sobre los endpoints que él mismo defina, con sus resultados documentados.

RNF12 Control de versiones en GitHub: Todo el proyecto debe alojarse en un repositorio de GitHub por equipo, en el cual debe evidenciarse la actividad de todos los integrantes mediante commits, ramas y pull requests.

RNF13 Mantenibilidad: El código debe seguir buenas prácticas de diseño, promoviendo bajo acoplamiento y alta cohesión.

RNF14 Portabilidad: Gracias a la contenerización, el sistema debe poder desplegarse en cualquier entorno compatible con Docker y Kubernetes sin cambios en el código fuente.

RNF15 Tecnologías permitidas: Únicamente podrán emplearse los lenguajes, frameworks, bases de datos y demás herramientas vistas y trabajadas durante las clases del curso.

RNF16 Uso documentado de IA: El proceso de desarrollo debe evidenciar, mediante las bitácoras correspondientes, el uso crítico y comprendido de herramientas de inteligencia artificial.

RNF17 Comunicación entre microservicios: Toda la comunicación entre microservicios se realizará mediante API REST sobre HTTP.

RNF18 Aislamiento de fallos: El sistema deberá permitir que la falla de un microservicio no crítico, como Reportes y Dashboard, no impida la ejecución de las operaciones principales de los demás microservicios que no dependan directamente de dicho servicio.

RNF19 Configuración independiente por empresa: El microservicio de Configuración debe soportar parámetros independientes para cada empresa cliente, garantizando que los ajustes de una empresa no afecten a otra.

RNF20 Trazabilidad e integridad de registros: Los registros del balance diario y de los movimientos de inventario deben conservar su trazabilidad e integridad una vez generados. Las correcciones deberán realizarse mediante nuevos registros de ajuste, conservando el registro original para permitir auditorías posteriores.

6 Bitácora de uso de IA

Fecha	Integrante	Herramienta	Tarea apoyada	Prompt (resumen)	Interv. humana	Resultado / aprendizaje
25/09/2026	Jasmin Gutierrez	chatGPT	Elaboración microservicios	Qué es un (dominios/bounded contexts).	Bajo	Dominio: es un área o problema específico del negocio que el sistema debe resolver. Bounded Context: es el límite dentro del sistema que define las responsabilidades, conceptos y reglas de un dominio.
25/09/2026	Jasmin Gutierrez	chatGPT	Elaboración microservicios	Cuál es la diferencia entre REST y mensajería asíncrona.	Bajo	REST: un microservicio le hace una solicitud a otro y espera su respuesta inmediata. Mensajería asíncrona: un microservicio envía un mensaje o evento y continúa, sin esperar una respuesta inmediata.
25/09/2026	Jasmin Gutierrez	Gemini	Investigación de servicios corporativos	Dame nombres de software de sistema de inventario que trabaje en modalidad corporativo, dame las ventajas, funcionalidad, y debilidades por favor.	Alta	Aprendí cómo los sistemas corporativos permiten controlar inventarios, compras, ventas, proveedores y existencias de diferentes áreas o sucursales.
25/09/2026	Jasmin Gutierrez	chatGPT	Creación de microservicios	Es necesario tener un microservicio de contabilidad al querer generar reportes de ventas y cierres de caja por local y ventas en web.	Alta	Aprendí que no es necesario crear un microservicio de contabilidad, ya que los reportes de ventas y cierres de caja requeridos pueden gestionarse mediante el microservicio de Balance Diario. La contabilidad implica procesos adicionales que están fuera del alcance del proyecto.
25/09/2026	Jasmin Gutierrez	chatGPT	Revisión de la propuesta de arquitectura	Se presentó la tabla de microservicios y se solicitó identificar posibles inconsistencias o mejoras en su organización.	Alta	Dijo que la separación estaba bien, pero que aclarara mejor Reposición y Balance Diario.
25/09/2026	Jasmin Gutierrez	chatGPT	Investigación de servicios de inventarios	Se solicitó información para comparar sistemas como Odoo, Alegra y otros sistemas de gestión de inventarios.	Alta	Se obtuvieron funcionalidades para construir una comparación. El equipo seleccionó la información pertinente y realizó la comparación de acuerdo con los criterios solicitados en la actividad.
22/09/2026	Amanda Galeano	Claude (Anthropic)	Armar la estructura base del documento	Pedí una estructura del documento con todas las secciones que pide el enunciado, para no olvidarnos de nada importante.	Medio	Aprendí que es más fácil partir de una estructura ya ordenada y luego llenarla en equipo, en vez de armar el documento desde cero cada uno por su lado.

Fecha	Integrante	Herramienta	Tarea apoyada	Prompt (resumen)	Interv. humana	Resultado / aprendizaje
23/09/2026	Amanda Galeano	Claude (Anthropic)	Redactar una primera versión de la definición del problema	Le di las ideas sueltas que teníamos como equipo (inventario, ventas en tienda y web, reposición automática, panel personalizable) y pedí que las organizara en texto.	Medio	Faltaba aclarar el tema de precios de venta y se lo señalé; también decidí junto con mi equipo qué quedaba dentro y fuera de nuestro proyecto.
25/09/2026	Amanda Galeano	Claude	Evaluar si conviene fusionar los microservicios de Balance Diario y Reportes/Dashboard	Pregunté si sería mejor combinarlos ya que son un poco similares.	Bajo	Entendí que no era buena idea ya que reportes agrega información de casi todos los demás microservicios, mientras que balance diario solo tiene un tipo de dato específico (ingresos/egresos); si los combináramos, este terminaría dependiendo de todos los demás servicios.
25/09/2026	Amanda Galeano	Claude	Redactar la justificación técnica de microservicios frente a monolito	Pedí ayuda para conectar los criterios técnicos (escalabilidad, mantenibilidad, etc.) con las necesidades reales de nuestro proyecto, no como argumento genérico.	Bajo	Entendí que en un monolito cualquier cambio pequeño obliga a volver a desplegar todo el sistema completo, mientras que en microservicios cada servicio se actualiza por separado sin tocar los demás.
25/09/2026	Kevvin Hernandez	Claude	Redactar de mejor manera los nuevos requerimientos funcionales y no funcionales, además de pedir opinión sobre mejora y cambio de los requerimientos	Ayúdame a redactar mejor estos requerimientos funcionales y no funcionales. Al terminar revisa si todos los requerimientos son necesarios y cómo se podrían mejorar o cambiar.	Medio	El resultado fue un documento mejor redactado, con lenguaje más formal, con sus requerimientos mejor expresados y también con nuevos y mejores requerimientos especialmente para los microservicios.
25/09/2026	Kevvin Hernandez	Claude (Anthropic)	Conversión del documento del proyecto a código LaTeX	Pasámelo todo a código LaTeX con el mismo tipo de letra del documento Word.	Medio	Aprendí a identificar la fuente y la estructura real del documento (portada, tablas, listas de RF/RNF) para poder pedirle a la IA una transcripción fiel a LaTeX, revisando después que el contenido técnico (tablas de microservicios, RF y RNF) coincidiera exactamente con lo que ya teníamos en Word.